

Cahier des Charges

Internal Asset Exchange Platform

Plateforme interne d'échange et de transfert d'actifs

Méthodologie : Agile / Scrum — Cycle de 30 jours (6 Sprints)
Technologies : PHP (MVC custom), MySQL, Bootstrap 5, JavaScript
Document de spécification fonctionnelle et technique

Version 1.0 — Juin 2026

Table des matières

1. Présentation du projet.....	3
2. Périmètre du projet.....	3
3. Acteurs et rôles.....	4
4. Architecture technique.....	4
5. Modèle de données.....	5
6. Spécifications fonctionnelles détaillées.....	6
7. Exigences de sécurité.....	7
8. Planification Agile (Sprints).....	8
9. Critères d'acceptation (Definition of Done).....	8
10. Déploiement et environnement.....	9

1. Présentation du projet

1.1 Contexte

Au sein des entreprises, il est fréquent que du matériel informatique ou du mobilier de bureau reste inutilisé dans certains départements alors que d'autres départements ont besoin de ce même type d'équipement. Cette situation entraîne un gaspillage budgétaire, une accumulation d'objets non valorisés et l'achat de nouveaux équipements alors que des ressources existantes pourraient être réaffectées.

1.2 Objectif du projet

Le projet consiste à développer une plateforme web interne permettant à une entreprise de gérer et transférer le matériel inutilisé entre ses différents départements. La plateforme doit permettre à plusieurs entreprises distinctes (multi-tenant) d'utiliser le même système, chacune disposant d'un espace de travail totalement isolé.

1.3 Bénéfices attendus

- Réduction des coûts d'achat de nouveaux équipements
- Meilleure visibilité sur les ressources matérielles disponibles
- Traçabilité complète des transferts d'actifs (qui a demandé, qui a approuvé, quand)
- Réduction du gaspillage et impact environnemental positif (réemploi)
- Aucune installation technique requise : chaque entreprise crée son espace en quelques clics

2. Périmètre du projet

2.1 Fonctionnalités incluses ("Must Have" / "Should Have")

- Création d'un espace de travail (workspace) par entreprise, avec génération d'un code unique
- Authentification sécurisée (email + mot de passe + code entreprise)
- Gestion des rôles : Team Leader (responsable) et Team Member (membre)
- Ajout, consultation, filtrage et suppression d'actifs (avec image)
- Demande de transfert d'un actif entre départements
- Approbation / rejet des demandes de transfert par le Leader
- Historique complet des transferts effectués et calcul des économies réalisées
- Gestion des membres (ajout par le Leader, génération de mot de passe temporaire)
- Changement de mot de passe en self-service
- Tableau de bord avec statistiques (actifs disponibles, transférés, économies)
- Notifications par email et messages popup pour chaque action importante

2.2 Fonctionnalités explicitement hors périmètre ("Won't Have")

- Transferts inter-entreprises (chaque entreprise reste totalement isolée)
- Dépréciation automatique / amortissement comptable des actifs
- Messagerie instantanée (chat en temps réel)
- Création dynamique de catégories personnalisées (la catégorie reste un champ texte libre)

3. Acteurs et rôles

Rôle	Description	Permissions principales
Invité (Guest)	Visiteur non authentifié	Créer une nouvelle entreprise (devient Leader)
Team Member	Employé membre d'un département	Voir/ajouter/supprimer ses actifs, demander un transfert, consulter l'historique, changer son mot de passe
Team Leader	Responsable de l'entreprise (créateur du workspace ou promu)	Toutes les permissions Member + ajouter des membres, approuver/rejeter les transferts, consulter les statistiques complètes

Règle métier clé : un membre peut supprimer ("undo") un actif uniquement s'il appartient au même département que l'actif ET que celui-ci est encore au statut "available".

4. Architecture technique

4.1 Stack technologique

Couche	Technologie
Backend	PHP 8.2 (architecture MVC personnalisée)
Base de données	MySQL (via PDO, requêtes préparées)
Frontend	HTML5, CSS3, Bootstrap 5, JavaScript
Serveur local	Apache (XAMPP)
Email	PHPMailer / fonction mail() native

4.2 Architecture MVC (Modèle-Vue-Contrôleur)

Le projet suit une architecture MVC personnalisée afin de séparer la logique métier, l'affichage et le contrôle des flux :

- Models (models/) : accès aux données via PDO (Company, User, Asset, TransferLog)
- Views (views/) : pages HTML/PHP affichées à l'utilisateur, organisées par module
- Controllers (controllers/) : logique applicative, validation, orchestration
- Core (core/) : classes de base (Database, Model, Controller, Router)
- Helpers (helpers/) : fonctions utilitaires transverses (session, authentification, sécurité)

4.3 Routage et point d'entrée unique

Toutes les requêtes passent par un contrôleur frontal unique (public/index.php). Un fichier .htaccess à la racine redirige toutes les URL vers ce point d'entrée, qui utilise une classe

Router pour dispatcher la requête vers le bon contrôleur et la bonne méthode, selon la méthode HTTP (GET/POST) et le chemin demandé (y compris les paramètres dynamiques de type {id}).

Une constante `BASE_URL` est définie de manière centralisée afin que tous les liens, formulaires et redirections restent cohérents quel que soit l'emplacement du projet sur le serveur.

4.4 Structure des dossiers

```
internal-asset-exchange/
  config/      -> database.php (parametres BD)
  controllers/ -> AuthController, AssetController,
                  TransferController, UserController
  models/      -> Company, User, Asset, TransferLog
  views/       -> auth/, assets/, transfers/, members/,
                  layouts/, dashboard.php
  public/      -> index.php (point d'entree), uploads/,
                  css/, js/
  core/        -> Router, Controller, Model, Database
  helpers/     -> SessionHelper, AuthHelper,
                  SecurityHelper
  sql/         -> database.sql (script de creation)
  .htaccess    -> redirection vers public/index.php
```

5. Modèle de données

La base de données repose sur 4 tables principales, avec un cloisonnement strict des données par entreprise (`company_id`) afin de garantir l'isolation multi-tenant.

5.1 Table companies

Champ	Type	Description
id	INT (PK)	Identifiant unique
name	VARCHAR(100)	Nom de l'entreprise
company_code	VARCHAR(50) UNIQUE	Code unique généré automatiquement (ex : ABC12345)
created_at	TIMESTAMP	Date de création

5.2 Table users

Champ	Type	Description
id	INT (PK)	Identifiant unique
company_id	INT (FK)	Référence vers l'entreprise
name	VARCHAR(100)	Nom de l'utilisateur
email	VARCHAR(100)	Email (unique par entreprise)
password	VARCHAR(255)	Mot de passe haché (password_hash)

role	ENUM	leader ou member
department	VARCHAR(100)	Département de l'utilisateur
created_at	TIMESTAMP	Date de création

5.3 Table assets

Champ	Type	Description
id	INT (PK)	Identifiant unique
company_id	INT (FK)	Entreprise propriétaire
added_by	INT (FK)	Utilisateur ayant ajouté l'actif
name / category	VARCHAR	Nom et catégorie (texte libre)
description	TEXT	Description détaillée
value	DECIMAL(10,2)	Valeur financière de l'actif
condition	VARCHAR(50)	État de l'actif (ex : good, used...)
image_path	VARCHAR(255)	Chemin de l'image uploadée
status	ENUM	available / pending_transfer / assigned
department	VARCHAR(100)	Département propriétaire

5.4 Table transfer_log

Champ	Type	Description
id	INT (PK)	Identifiant unique
company_id / asset_id	INT (FK)	Entreprise et actif concernés
requester_id / approver_id	INT (FK)	Demandeur et approbateur
from_department / to_department	VARCHAR(100)	Départements source et cible
asset_value	DECIMAL(10,2)	Valeur de l'actif au moment du transfert
status	ENUM	pending / approved / rejected
request_date / approval_date	TIMESTAMP	Dates de demande et de décision

Toutes les clés étrangères vers companies utilisent ON DELETE CASCADE, garantissant qu'une suppression d'entreprise entraîne automatiquement la suppression de toutes ses données associées.

6. Spécifications fonctionnelles détaillées

6.1 Création d'un espace de travail

1. Un invité accède au formulaire de création d'entreprise
2. Il renseigne : nom de l'entreprise, son nom, email, mot de passe, département
3. Le système génère un code entreprise unique (8 caractères alphanumériques)
4. L'utilisateur est automatiquement créé avec le rôle "leader"
5. Le code entreprise est affiché dans une popup et envoyé par email

6.2 Authentification

6. L'utilisateur saisit : email, mot de passe, code entreprise
7. Le système vérifie la correspondance et valide le mot de passe via `password_verify()`
8. Une session est initialisée (`user_id`, `role`, `company_id`, `department`)
9. L'utilisateur est redirigé vers un tableau de bord adapté à son rôle

6.3 Gestion des actifs

10. Tout utilisateur connecté peut consulter la liste des actifs disponibles de son entreprise (avec recherche et filtres par catégorie/département)
11. Tout utilisateur peut ajouter un actif avec image, valeur, état et description
12. La consultation détaillée affiche toutes les informations de l'actif
13. Un membre du même département peut supprimer ("undo") un actif disponible

6.4 Processus de transfert

14. Un membre consulte un actif disponible et clique sur "Demander un transfert"
15. Une entrée est créée dans `transfer_log` avec le statut "pending", et l'actif passe en "pending_transfer"
16. Le Leader reçoit une notification et consulte la liste des demandes en attente
17. S'il approuve : l'actif passe au statut "assigned", la demande passe à "approved", la date d'approbation est enregistrée
18. S'il rejette : l'actif redevient "available", la demande passe à "rejected"
19. Le demandeur reçoit un email l'informant de la décision

6.5 Historique et statistiques

- Liste de tous les transferts approuvés, avec filtres par département et par période
- Calcul automatique du total des économies réalisées (SUM des valeurs des actifs transférés)
- Tableau de bord affichant : nombre d'actifs disponibles, nombre d'actifs transférés, top 3 des actifs les plus chers, et (pour les Leaders) le nombre de demandes en attente

6.6 Gestion des membres et du mot de passe

- Le Leader peut ajouter un membre : un mot de passe temporaire est généré automatiquement et affiché + envoyé par email
- Tout utilisateur peut changer son mot de passe (ancien mot de passe requis, nouveau $\geq$ 6 caractères, confirmation)
- La session reste active après le changement de mot de passe

7. Exigences de sécurité

- Hachage des mots de passe via password_hash() / password_verify() — aucun mot de passe en clair
- Requêtes SQL exclusivement via des requêtes préparées PDO (protection contre les injections SQL)
- Échappement systématique des sorties via htmlspecialchars() (protection contre les XSS)
- Isolation multi-tenant stricte : toute requête est filtrée par company_id de la session active
- Contrôle d'accès basé sur les rôles (middleware AuthHelper::requireLogin() / requireRole())
- Blocage de la navigation arrière/avant vers des pages protégées après déconnexion (en-têtes no-cache)
- Protection contre l'accès direct aux dossiers internes (config, core, models...) via la structure des dossiers et le routage centralisé

8. Planification Agile (Sprints)

Le projet est découpé en 6 sprints sur un cycle total de 30 jours, selon la méthodologie Scrum.

Sprint	Durée	Objectif principal	Livrables clés
Sprint 0	Jours 1-4	Fondations & environnement	Base de données, architecture MVC, routage, configuration
Sprint 1	Jours 5-9	Authentification multi-tenant	Création d'entreprise, login/logout, tableaux de bord par rôle
Sprint 2	Jours 10-14	Gestion des actifs	CRUD actifs, upload d'images, recherche/filtres
Sprint 3	Jours 15-20	Système de transfert	Demandes, approbation/rejet, historique, économies
Sprint 4	Jours 21-25	Comptes & statistiques	Gestion des membres, changement de mot de passe, dashboard stats
Sprint 5	Jours 26-30	Polish & déploiement	Notifications, responsive design, sécurité finale, mise en ligne

8.1 Estimation Fibonacci

L'effort global du projet est estimé à environ 54 points (échelle de Fibonacci : 1, 2, 3, 5, 8), répartis sur 21 user stories couvrant les 6 épopées fonctionnelles (Foundation, Asset Management, Transfer System, User Management, Notifications & Polish, Technical & Deployment).

9. Critères d'acceptation (Definition of Done)

Une user story est considérée comme terminée si elle respecte l'ensemble des critères suivants :

- Le code respecte l'architecture MVC et a été testé localement sans régression
- Les opérations sur la base de données s'exécutent sans erreur
- Les entrées utilisateur passent par des requêtes préparées et les sorties sont échappées
- L'isolation des données entre entreprises est vérifiée (aucune fuite de données)
- Chaque page protégée vérifie la session active avant affichage
- Chaque action de modification de données déclenche un message de confirmation (popup et/ou email)

10. Déploiement et environnement

10.1 Environnement de développement

- Serveur local : XAMPP (Apache 2.4, PHP 8.2, MySQL)
- Base de données créée via import du script sql/database.sql dans phpMyAdmin
- Accès local via : `http://localhost/<dossier-projet>/public/`

10.2 Déploiement en production

- Hébergement compatible PHP/MySQL (ex : Hostinger, InfinityFree)
- Import du même script database.sql sur le serveur de production
- Mise à jour de config/database.php avec les identifiants de production
- Ajustement de la constante BASE_URL selon le chemin de déploiement (vide si à la racine du domaine)
- Vérification des droits d'écriture sur le dossier public/uploads/assets/
- Test final multi-entreprises sur l'environnement de production